中国科学技术大学

面向应用场景的虚拟机自动调优方法

(2025 年 9 月 - 2025 年 12 月)

虚拟机参数配置实证调研分析报告

张昱　课题组

系统软件与软件安全研究组 (S4Plus)

中国科学技术大学 · 计算机科学与技术学院

USTC-CS: S4Plus 张昱课题组

目 录

1 概述 1
1.1 项目概述 1
1.2 研究架构与任务划分 2
2 虚拟机配置参数的实证分析 4
3 信息提取 5
4 测试与执行 7
4.1 Generator 8
4.1.1 随机生成 8
4.1.2 搜索生成 9
4.2 MetricsCollector 9
4.3 PageExecutor 13
5 参数筛选与分析 14
5.1 Classifier 15
5.2 ImpactAnalyzer 17
5.2.1 数据收集 17
5.2.2 指标计算 17
5.2.3 参数分类 18
5.2.4 分类结果 18
6 总结与未来工作 19
A 影响值 >0.03 的参数集合 20

USTC-cs: SAPlus 张昱课题组

1 概述

1.1 项目概述

本项目是合作项目《面向应用场景的虚拟机自动调优方法》。本项目拟设计一套面向 JavaScript 虚拟机的高效自动调优框架，在配置项选取、搜索策略设计与调优部署方式之间建立协同机制，以提升系统整体的调优效能与适应性。该框架将在开源的 JavaScript 虚拟机 V8 上进行部分原型实现与验证。

预期成果：针对在线办公类 Web 页面实现页面端到端时延降低 5% 左右。

本项目的主要交付包含：

- 软件交付：支持混合调优方式的开源虚拟机软件原型。
- 文档交付：
  - 《虚拟机参数配置实证调研分析报告》：包含不同配置参数的作用、依赖关系的分析，程序特征对配置参数选择的影响的分析。
  - 《性能模型与混合调优框架设计报告》：包含性能模型设计文档；基于 v8 和 chromium 的混合调优架构的设计文档；技术创新说明书。
  - 《调优工具测试报告》：包含详细的测试方法、数据集或场景以及本调优工具的测试结果。

本项目共划分为四个阶段，其中每个阶段的任务划分和交付如图 1所示。

图 1: 项目阶段划分

项目第一阶段进展概况：

- 利用脚本，从 V8 源码中提取工 937 个可调整的参数，包括其类型、默认值、描述等信息；提取出参数之间的依赖关系共 363 个。
- 实现基于 V8 命令行工具 d8 和 Chromium 进行的实证测试框架，支持通过配置不同参数来执行 JS 代码以收集实证实验数据。
- 通过对 V8 的修改实现对代码特征的采集，目前共设计了 19 种不同的代码特征用于对特定场景的 JS 程序进行表征。
- 利用实证分析框架对参数进行进一步的分析和筛选。首先，利用 LLM 与人工分析结合的方式，将 937 个参数依据功能分为了 16 类；其次通过采集的实证数据量化了不同参数对性能的影响程度，并最终基于该结果筛选出 171 个不同的参数作为调优的目标参数。

1.2 研究架构与任务划分

图 2展示了本项目的总体设计架构。在 JavaScript 虚拟机中，JavaScript 代码通常以函数为基本单位进行解释或编译执行，并且随着函数热度的提升，其执行方式将从解释执行逐步过渡到基础编译（无优化或少量优化）再到优化编译（基于运行时 PGO 信息进行激进优化）。除了编译器与解释器外，虚拟机还包含垃圾回收等多个关键运行时组件。

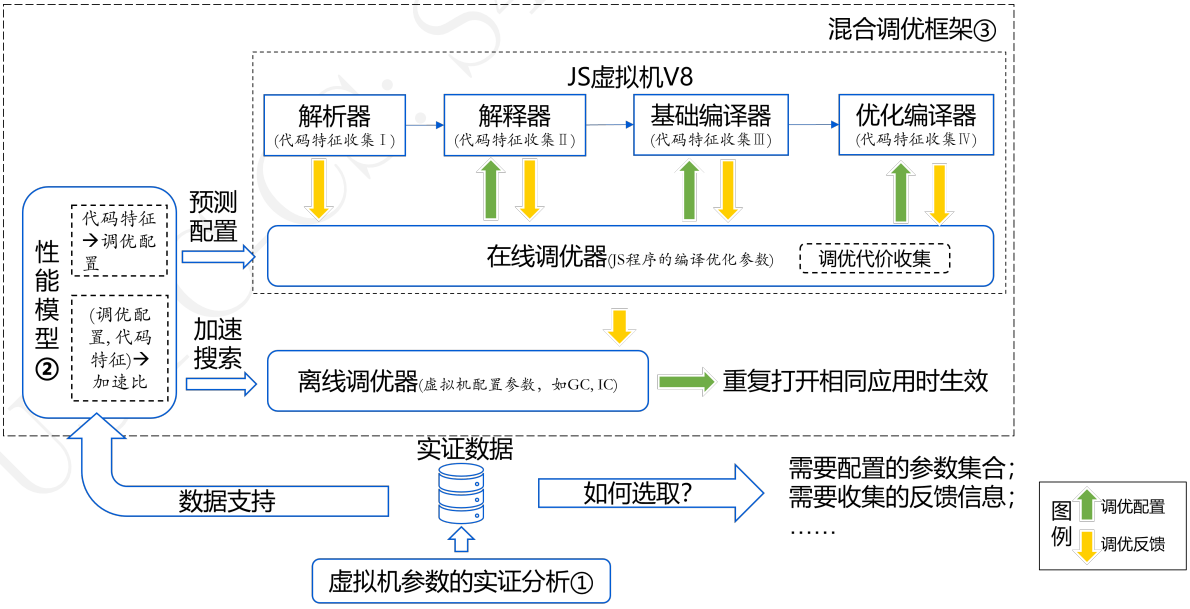

图 2: 整体研究架构

基于此，本项目拟从两个层次对虚拟机进行调优。其一，在方法/函数粒度上，针对不同编译阶段为各函数搜索（近似）最优的编译配置，实现更精细的优化控制；其二，在程序粒度上，根据整体应用特征对虚拟机各运行时子系统（如 GC）进行参数调优，从而提升全局执行效率。为构建高效的 JavaScript 虚拟机自动调优框架，本项目计划从以下三个方面展开系统性研究：

虚拟机参数配置实证分析与搜索空间建模。首先对虚拟机已有的参数配置项进行系统梳理与依赖分析，识别参数对运行时模块的影响范围，并构建函数粒度与虚拟机粒度的双层配置集合。基于参数依赖性构建配置图谱，以支持搜索空间缩减与优化路径约束。同时，结合典型终端应用场景开展实证分析，量化参数对应用性能的影响，提取关键程序特征与配置效能间的关联，为调优反馈设计和性能建模奠定基础。另一方面，通过实证分析的方式收集用于进行模型训练的数据集。

性能模型训练与调优器的搜索效率提升。面向资源受限的终端环境，项目构建两类性能预测模型：其一为训练（调优配置，代码特征）加速比的性能模型，它接受优化配置与程序特征，输出性能指标的联合建模，用于快速评估候选配置；其二为训练代码特征调优配置的性能模型，由程序特征直接预测最优配置的映射模型，用于在线阶段的调优引导。两类模型协同支持高效搜索，显著降低调优开销，并提升自动化配置推荐的适应性与鲁棒性。

离线-在线混合的调优架构设计。本项目将探索离线与在线调优路径的有机融合：离线阶段以应用运行反馈为基础，借助训练模型在空闲时完成配置优化，提升后续启动性能；在线阶段则结合虚拟机多级编译特性，设计逐层反馈机制，实现函数级的动态优化。为缓解端侧算力压力，项目进一步提出端云协同方案，将高成本搜索任务迁移至云侧执行，构建轻量高效、可落地的调优架构。

USTC-CS SMPL 收集整理组

2 虚拟机配置参数的实证分析

现有将机器学习或搜索算法引入自动调优的工作，通常依赖一系列关键前提假设。例如：存在一组对性能或能耗等指标具有显著影响的可调参数；可以通过一组程序特征对不同 JavaScript 程序进行有效表征；参数配置与程序特征之间存在可学习的映射关系。然而，在 JS 虚拟机这一高度动态的执行环境中，这些前提往往并未经过充分验证。JavaScript 语言本身的动态类型、隐式转换、原型链机制以及高度依赖运行时行为的执行模式，使得虚拟机参数的影响路径更加复杂，也更难通过经验或直觉进行判断。

具体而言，一项面向 JS 虚拟机的自动调优工作通常需要解决多个相互耦合的问题：首先是目标参数的选择，即哪些虚拟机参数在实践中真正会对性能、启动时间或内存占用等指标产生可观影响；其次是程序特征的选择，需要回答“哪些静态或动态特征能够区分不同 JavaScript 程序，并与参数调优效果相关”；此外还涉及数据集的构建与选取，以确保实验结论具备代表性和外推能力；如果进一步引入搜索空间剪枝或分层调优策略，则还需要理解参数之间的依赖关系，以及参数与程序特征之间的交互效应。

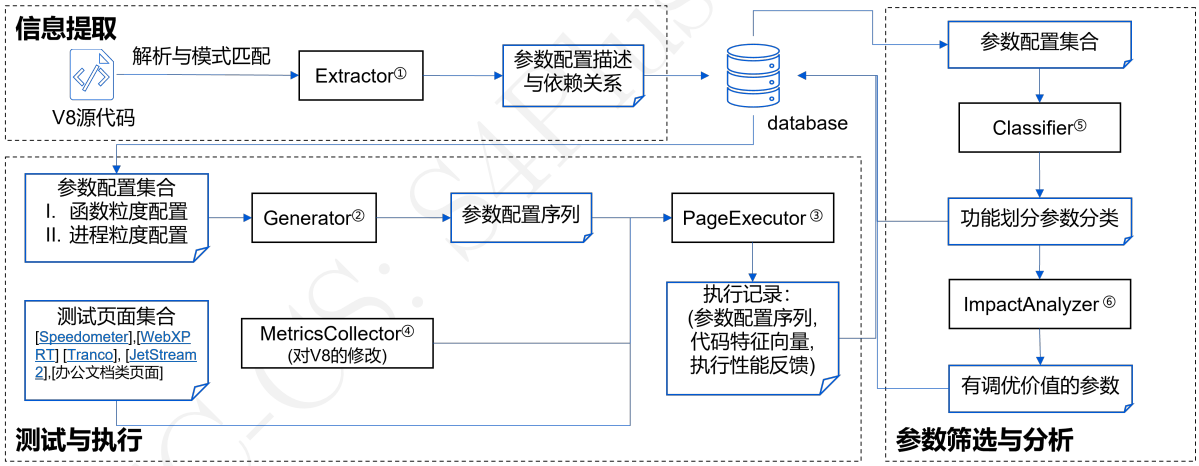

图 3: 实证分析研究框架

为此，我们设计如图 3所示的研究框架来开展分析与讨论。该框架共分三个部分：

- **信息提取**。在这一部分中，将从 V8 虚拟机的源码出发，从中提取和参数相关的基础信息（如参数名称、类型、默认值、功能描述等）、参数依赖与约束关系。
- **测试与执行**。这一部分主要用于通过不同的参数执行现实世界中不同的 JS 应用程序或网页应用，并收集如程序特征、性能结果等数据，为进一步的分析提供实证和数据基础。它主要包括：

  - Generator。该工具将基于参数配置集合来生成一个可用的参数配置序列。一方面，它要求生成的参数取值是合法的；另一方面，它要求生成的参数是满足依赖和约束关系的。
  - MetricsCollector。该工具用于在虚拟机内部收集一些程序相关的元数据，如程序特征以及一些细粒度的性能表现。
  - PageExecutor。该工具用于在特定的参数下执行目标程序并收集性能反馈。
- **参数筛选与分析**。这一部分用于驱动“测试和执行”框架对参数进行不同维度的分析和筛选。
  - Classifier。该工具用于对参数按功能进行分类划分，可以基于功能进一步对参数做筛选。
  - ImpactAnalyzer。该工具通过调用“测试与执行”框架收集实证信息，并基于实证信息来分析不同参数对性能的影响，最终筛选出有调优价值的参数。

3 信息提取

本小节将介绍如何从 V8 源码和信息中提取参数配置序列的相关信息。

在 V8 的源代码中，所有的参数以及参数之间的依赖和约束关系在src/flags/flag-definitions.h文件中通过宏的形式被定义。代码清单 1展示了用于定义参数的一些宏以及相关的示例。其中，通过 `DEFINE_BOOL`和`DEFINE_INT`等不同的宏可以定义不同类型的参数，其信息包括：类型、对应的 C 类型、名称、默认值以及参数描述。例如，代码清单 1的第 8 行定义了一个布尔类型的参数`use_strict`，它默认值为 false，它的功能用于强制启用 strict 模式。同理，`use_strict`参数为 int 类型，默认值为 3000，它用来控制函数经过多少次调用可以被 Turbofan 优化。

代码清单 1: flag-definitions.h 中的参数定义

```
1  #define FLAG_FULL(ftype, ctype, nam, def, cmt) \
2    {Flag::TYPE_##ftype, #nam, &v8_flags.nam, &FLAGDEFAULT_##nam, cmt, false},
3  #define FLAG FLAG_FULL
4  #define DEFINE_BOOL(nam, def, cmt) FLAG(BOOL, bool, nam, def, cmt)
5  #define DEFINE_INT(nam, def, cmt) FLAG(INT, int, nam, def, cmt)
6  #define DEFINE_STRING(nam, def, cmt) FLAG(STRING, const char*, nam, def, cmt)
7
8  DEFINE_BOOL(use_strict, false, "enforce␣strict␣mode")
9
10 DEFINE_INT(invocation_count_for_turbofan, 3000,
11            "invocation␣count␣required␣for␣optimizing␣with␣TurboFan")
```

另一方面,在上述文件中还定义了参数之间的依赖关系。宏DEFINE_VALUE_IMPLICATION定义了参数 thenflag 在 cond 成立的时候会被设置为 value。基于此，V8 扩展定义了很多不同类别的依赖关系。例如代码清单 2中给出了两种依赖关系,其中第 10 行表示参数maglev_as_top_tier在被设置时,maglev需要为 true;第 11 行则表示参数maglev_as_top_tier在被设置时，maglev需要为 false。表 1则展示了 V8 中所有不同的依赖关系，其中包括真依赖、假依赖、弱依赖等共 5 种不同的依赖关系。

代码清单 2: flag-definitions.h 中的参数依赖定义

```
1  #define DEFINE_VALUE_IMPLICATION(cond, thenflag, value)                    \
2    changed |= TriggerImplication(cond, #cond, &v8_flags.thenflag, #thenflag, \
3                                  value, false);
4
5  #define DEFINE_IMPLICATION(cond, thenflag) \
6    DEFINE_VALUE_IMPLICATION(cond, thenflag, true)
7
8  #define DEFINE_NEG_IMPLICATION(cond, thenflag) \
9    DEFINE_VALUE_IMPLICATION(cond, thenflag, false)
10
11 DEFINE_IMPLICATION(maglev_as_top_tier, maglev)
12 DEFINE_NEG_IMPLICATION(maglev_as_top_tier, turbofan)
```

表 1: V8 中依赖不同依赖的类型

依赖关系 (src, target, value)	目标参数类型	描述
真依赖	bool	当 src 为 true 时，target 为 true
	value	当 src 为 true 时，target 为 value
假依赖	bool	当 src 为 false 时，target 为 true
	value	当 src 为 false 时，target 为 value
弱-真依赖	bool	当 src 为 true 时，target 为 true；会被其他的依赖覆盖

我们可以形式化地定义参数的集合和取值空间，以便在后文中介绍。

定义 1 (参数空间) 设 $P = \{p_1, p_2, ..., p_n\}$ 为 *V8* 虚拟机暴露的可调参数集合，其中 n 表示参数的总数量。对于每一个参数 $p_i \in P$，其拥有特定的数据类型 T_i（如 *Boolean, Integer, String* 等）以及对应的取值域 D_i。整个参数空间的笛卡尔积可以表示为 $D = D_1 \times D_2 \times ... \times D_n$。同时,我们可以将参数之间的依赖关系定义为集合 $R = \{r_1, r_2, ..., r_m\}$，其中每个依赖关系 r_j 可以表示为一个三元组 $\langle p_{src}, p_{target}, value \rangle$。根据 *V8* 源码中的定义，主要包含以下几种依赖类型：

USTC-CS: UPlus 张昱课题组

- **真依赖（*True Implication*）**：当源参数 p_{src} 为真时，目标参数 p_{target} 被强制设置为特定值（通常为真或指定数值）。
- **假依赖（*False Implication*）**：当源参数 p_{src} 为假时，目标参数 p_{target} 被强制设置为特定值。
- **弱-真依赖（*Weak True Implication*）**：类似于真依赖，但该依赖关系具有较低的优先级，可能会被其他强依赖或用户显式配置所覆盖。

最终，我们使用 Python 实现了一个脚本，它通过解析对应版本 V8 源码中的 flag-definitions.h 文件中的定义，来解析所有的参数描述以及参数依赖关系。然而直接从宏定义中解析信息还面临着一些由于条件编译而导致的错误识别问题。例如，同一个参数可能在不同的平台下有着不同的定义方式，在 C 头文件中会通过条件编译`#ifdef`等形式来控制。为了减少由于条件编译带来的错误识别问题，在实现中会通过固定特定平台（如 amd64/linux）相关的编译选项来排除上述问题。此外，我们将解析出来的参数信息保存在数据库中以支持基于该结果的分析和进一步的调优。在这个过程中，我们共记录了 937 个被定义的有效参数，363 个不同参数之间的依赖和约束关系。

4 测试与执行

在完成参数的基础信息的收集之后，将基于参数的基础信息实现“测试与执行”框架。“测试与执行”框架用于通过 V8 的命令行程序 d8 或 Chromium 在实际的应用程序中以不同的参数配置项进行测试与信息收集，从而使能进一步的分析工具等。

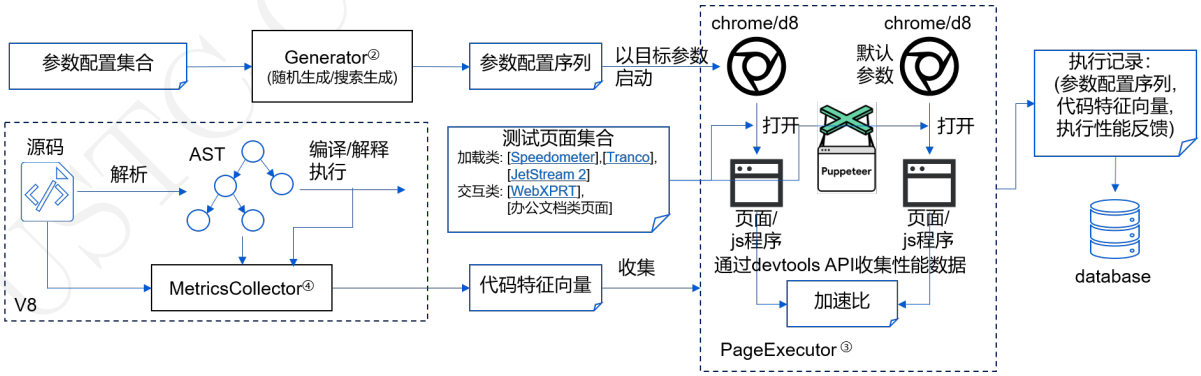

图 4: 测试与执行框架

图 4展示了“测试与执行”框架的总体设计。该框架分为三个组件：用于生成参数

配置序列的 Generator，用于进行执行和信息收集的 PageExecutor 以及用于进行运行时元信息收集的 MetricsCollector。

4.1 Generator

Generator 部分负责基于节 3中记录的参数信息进行参数配置的生成。

基于参数集合 P，一个具体的参数配置 c 可以被表示为向量形式 $c = \langle v_1, v_2, ..., v_n \rangle$，其中 $v_i \in D_i$ 表示参数 p_i 的具体取值。然而，并非所有在 D 中的配置都是有效的。V8 参数之间存在着复杂的依赖约束关系（例如，开启某个优化选项可能隐含地要求开启另一个基础选项）。在进行生成时，不仅要保证参数本身是有效的，还要保证参数满足依赖关系。

基于上述定义，Generator 的主要功能就是要生成一个或多个参数配置，使得每个参数配置 c 都满足：

- **域约束（Domain Constraint）**：$\forall i \in [1, n], v_i \in D_i$，即每个参数值都在其定义域内。
- **依赖约束（Dependency Constraint）**：$\forall r = \langle p_{src}, p_{target}, value \rangle \in R$，若配置 c 中 p_{src} 的取值满足触发条件，则 p_{target} 必须被设置为 $value$。即生成的配置必须满足所有定义的参数依赖关系。

针对不同的需求，设计了两种不同的参数配置生成方式：随机生成和搜索生成。前者基于随机的形式生成一个可用的参数配置，后者则基于现有的调优器中的搜索器生成一系列针对目标程序的优化参数配置。

4.1.1 随机生成

随机生成策略采用均匀采样的方法来探索参数空间。对于参数集合 P 它的配置生成策略为：

1. $c \leftarrow c_{default}$，其中 $c_{default}$ 表示参数集合 P 的默认配置，即每个参数 p_i 均取默认值。
2. 从参数集合 P 中随机选取 m 个参数组成新的集合 P'。
3. 对于 P' 中的每个参数 $p_i \in P'$，尝试在 c 中对 p_i 的取值进行变异：
    - 若存在依赖关系 $r = \langle p_{src}, p_i, value \rangle \in R$，则不尝试对 p_i 的值进行变异，直接将其值设置为 $value$。

USTC-CS-SPIRIS 张量课题组

- 若存在依赖关系 $r = \langle p_i, p_{target}, value \rangle \in R$，且 $p_{target} \in P'$ 已经在 p_i 之前被变异过。此时意味着，p_i 在进行变异之前的取值已经满足约束关系，再进行变异会导致约束关系被破坏，因此不尝试对 p_i 的值进行变异。
- 否则，依据参数 p_i 不同类型，从其合法的取值空间 D_i 中以等概率随机选取一个值 v_i。

4. 上述步骤得到的 c 即为一个有效的参数配置。

通过这种方式，Generator 生成一个完整的参数配置向量 c。该方法能够快速生成大量多样化的配置样本，用于构建初始的训练数据集，帮助模型学习参数与性能之间的非线性映射关系。

4.1.2 搜索生成

搜索生成策略基于现有的调优器中的搜索器生成一系列针对目标程序的优化参数配置。它的配置生成策略为：

1. 从参数集合 P 中随机选取 m 个参数组成新的集合 P'。
2. 将 P' 定义在调优器的搜索空间中。
3. 让调优器在设定的目标场景下进行调优搜索，在这个过程中记录每一个迭代所生成的参数配置 c。

实际上，随机生成的随机性会导致大多数生成的参数配置在实际场景中的执行性能较差，如果将这些数据用于训练性能预测模型等，会由于缺乏正样本导致模型的准确性和泛化能力较差。因此，为了弥补这一问题，我们通过搜索生成的方式可以得到一系列有效的且在目标场景较优的样本，从而生成较高质量的数据集。

代码清单 3展示了一个使用 OpenTuner 基于 d8 进行调优搜索的示例。基于 Python 中的 opentuner 库，可以很轻松的调用它进行调优，需要在代码清单 3中的 manipulator 方法内定义出所有参数的搜索空间，即将参数 P' 和它们的依赖关系定义进去；同时需要在 run 方法中，定义如何在目标场景下执行参数配置 c（即代码中的 config），并返回执行结果。

4.2 MetricsCollector

本节将介绍如何通过修改 V8 以进行特征向量的提取。在代码特征工程中，特征的选取对于构建高精度的性能预测模型至关重要。一般来说，选取的代码特征应该具备以下几个显著特点：首先是代表性，即特征能够准确反映程序的运行时行为和资源使

代码清单 3: OpenTuner 示例代码

```
class V8TunerGen(MeasurementInterface):
  ...
  def manipulator(self):
      manipulator = ConfigurationManipulator()
      # config_generator为我们定义的用于表述参数空间的对象
      for param in self.config_generator.parameter_space:
          if param.is_bool():
              manipulator.add_parameter(BooleanParameter(param.name, depends=
        getDependency(param)))
          elif param.is_int():
              manipulator.add_parameter(IntegerParameter(param.name, param.min,
         param.max, depends=getDependency(param)))
      return manipulator

  def run(self, desired_result, input, limit):
      cfg = desired_result.configuration.data
      config = {}
      for param in self.config_generator.parameter_space:
          val = cfg[param.name]
          config[param.name] = val
      # runner为我们定义的执行器
      score = self.runner.run(config)
      return Result(time=score)
```

用模式，有效区分不同类型的计算负载；其次是相关性，特征应当与虚拟机的关键性能指标（如执行时间、内存占用等）或调优参数的敏感度呈现出较强的统计关联；再次是低开销，特征数据的采集过程不应引入过高的运行时损耗，以确保性能测量的准确性；最后是稳定性，即在重复运行中特征值应保持一致，减少噪声对模型训练的干扰。

实际上，特征工程一直是一个学术界中难以完美解决的问题，特征的选取通常更依赖于经验和直觉，并且对于不同的引用目标，特征工程的侧重点和方法也会有所不同。尤其是针对通用应用程序进行的特征工程，只有结合场景和实践才能找出一个合适的特征工程方法。因此在本阶段，我们将主要从一些已有的工作出发，基于他们的特征选取进行修改，并完成特征提取框架的搭建，以便一方面验证已有工作的效果，另一方面后续基于我们的目标进行进一步的调整。

表 2展示了我们的特征向量集合，它一部分来自于 [1]，我们在此基础上添加了部分其他特征。特征共分为三个不同的层次：抽象语法树 (AST) 层次、源码 (SRC) 层次以及硬件 (HW) 层次。在当前实验中，我们主要在固定的平台进行实验和测试，因此暂且先忽略硬件层次的特征。而 AST 和源码层次的特征，将通过修改 V8 以实现提取。

USTC CS SAPlus 张昱课题组

表 2: 特征向量集合

名称	描述	层次
ASTNodes	number of ast nodes	AST
Functions	number of functions	AST
Branches	number of branches	AST
Loops	number of loops	AST
Symbols	number of symbols	AST
Operators	number of operators	AST
CmpOperators	number of comparision operators	AST
Parameters	number of parameters	AST
ASTDepth	maximum AST depth	AST
Cyclomatic	cyclomatic complexity of the program	AST
CyclomaticAvg	average cyclomatic comlexity of functions	AST
Cache	size of cache (MB)	HW
CPU	CPU frequency (GHz)	HW
RAMFrqc	RAM frequency (GHz)	HW
RAMSize	size of RAM(GB)	HW
RAMChannels	number of RAM channels in Soc	HW
LOC	lines of code	source
CodeSize	Bytes of code	source
ParseSize	Bytes of parsed code	source

V8 会在获取需要执行的 JavaScript 文件后，经解析器解析产生 AST，部分重要的代码特征可以在这一过程中获取。更加具体地来说，V8 源代码目录 src 下的 parsing 和 ast 两个目录中的部分文件与解析阶段完成的静态代码特征提取密切相关，前者包含了 V8 的代码解析模块，后者包含 AST 结构的定义。而ast/ast.cc 和 ast/ast.h 中描述了每种 AST 节点的结构。ast/ast-traversal-visitor.h 中给出的 `AstTraversalVisitor` 基类能够从指定的根节点开始，遍历地访问 AST 中的每一个节点。因此，AST 层次的特征信息可以通过定义新的`AstTraversalVisitor`子类来进行 AST 遍历并获取。

代码清单 4展示了所实现的 AST 层次的特征提取器的部分代码示例。我们通过定义不同的 AST 节点的访问规则，在遍历 AST 的过程中记录不同的特征信息。例如，每当有一个分支语句时，累加用于记录分支数的变量和用于记录圈复杂度的变量。

此外，parsing/parser.cc 中的 `ParseProgram` 函数是程序解析的调用入口，其中包括扫描 JavaScript 源代码形成词法单元序列、由语法分析器产生 AST 和一些后处理工作。在 `ParseProgram` 中，通过在 AST 生成之后插入对特征提取器的调用，即可在 V8

USTCCS SAPlus 张量课题组

代码清单 4: AST 代码特征提取示例

```
class FeatureVectorAstVisitor final
    : public AstTraversalVisitor<FeatureVectorAstVisitor> {
    void FeatureVectorAstVisitor::VisitIfStatement(IfStatement* stmt) {
        stk.top().cyclomaticComplexity++;
        stk.top().numBranches++;
        AstTraversalVisitor<FeatureVectorAstVisitor>::VisitIfStatement(stmt);
    }
    ...
    void VisitBlock(Block* stmt);
    void VisitSwitchStatement(SwitchStatement* stmt);
    void VisitTryCatchStatement(TryCatchStatement* stmt);
    void VisitConditional(Conditional* expr);
    void VisitConditionalChain(ConditionalChain* expr);
    void VisitBreakStatement(BreakStatement* stmt);
    void VisitContinueStatement(ContinueStatement* stmt);
    void VisitFunctionLiteral(FunctionLiteral* expr);
    void VisitCompareOperation(CompareOperation* expr);
    void VisitBinaryOperation(BinaryOperation* expr);
    void VisitNaryOperation(NaryOperation* expr);
    void VisitUnaryOperation(UnaryOperation* expr);
    void VisitCountOperation(CountOperation* expr);
    void VisitAssignment(Assignment* expr);
    void VisitVariableProxy(VariableProxy* expr);
    void VisitVariableDeclaration(VariableDeclaration* decl);
    void VisitFunctionDeclaration(FunctionDeclaration* decl);
    bool VisitNode(AstNode *node);
    ...
}
```

解析代码的过程中获取其特征。

值得注意的是，在 V8 的 AST 结构中，每个代码文件的全部内容会被当作一个最外层函数的函数体，该函数对应节点正是 AST 的树根。因此，上述函数粒度的统计方法能够涵盖一个文件中的所有内容。

还有一些源码级别代码特征是无法在 AST 上获取的，例如代码行数、代码体积大小，这些指标需要直接从 JavaScript 源代码中收集。此外，网页 JavaScript 代码的特点是其中有许多和执行性能密切相关的 DOM 操作，该指标也应作为代码特征的重要指标纳入考量，我们会在后续的工作中尝试找到一种能低开销记录该特征的方式。

USTC CS. SPA 实验课题组

4.3 PageExecutor

PageExecutor 是用于执行测试并收集结果的工具。它主要支持以不同的参数启动目标测试，并将测试结果记录在数据库中。

我们的目标测试场景可以分为三类：

- **纯 JS 程序**。此类程序可以直接通过 V8 的命令行工具 d8 执行，主要包含三类 JS 数据集kraken, octane以及sunspider。
- **网页加载场景**。此类场景主要是针对一些热门的网页的首页加载时延的测试，即从打开目标页面开始到目标页面完成加载所花费的时间。目标页面包括tranco数据集中的筛选的部分页面以及一些其他用于测试网页性能的页面如Speedometer, JetStream。
- **交互类网页场景**。此类场景主要针对需要用户交互之后的操作时延进行测试，如一些办公文档类页面用户进行表格、字号等调整后，页面重新渲染所花费的时间。这包含我们的目标场景如腾讯文档以及一些其他的用户交互测试集如WebXRT

针对第一类场景，可直接通过 d8 命令行工具进行执行和测试即可。而针对后两类场景，需要实际地打开浏览器页面，进行相应的交互操作，通过页面变化情况来判断是否完成加载，并最终收集相关测试结果。这种场景的自动化测试需要借助浏览器的驱动实现。我们选择使用Puppeteer来帮助实现 PageExecutor。

Puppeteer 是一个 Node.js 库，它提供了一套高级 API 来通过 DevTools 协议控制 Chrome 或 Chromium。Puppeteer 默认以无头（Headless）模式运行，但也可以配置为在完整的（非无头）Chrome 或 Chromium 中运行。通过 Puppeteer，可以模拟用户在浏览器中的各种操作，如点击、输入、页面导航等，同时还可以捕获页面的性能数据、截图以及生成 PDF 等。此外，通过 Puppeteer，还可以设置 Chromium 的启动参数，通过这一参数允许我们将生成出来的参数配置传递给 V8。

基于 Puppeteer 实现的 PageExecutor 针对后两种场景的执行流程为：

1. 基于 Generator 生成的参数配置启动 Chromium.
2. 打开目标测试页面。
3. 执行用户交互操作（如果需要的话）。
4. 等待页面加载完成。对于网页加载场景，使用 Largest Contentful Paint (LCP) 事件来标志页面加载的完成，并基于此进行性能数据的记录；而对于交互类的页面场景，由于没有固定的事件标识，故通过判断页面是否稳定不发生变化来判断加载完成。具体来说，当连续 3 帧页面不再发生变化时，认为页面加载完成。

5. 利用 tracing 的方式来获取页面加载时延等性能指标。
6. 关闭 Chromium，通过 MetricsCollector 记录的内容收集和汇总特征向量，并存入数据库。

代码清单 5展示了执行单个页面并进行信息收集的部分代码示例。

代码清单 5: PageExecutor 部分代码示例

```
async function runSingleExperiment(paramGroup, targetUrl) {
    # 启动浏览器
    browser = await puppeteer.launch({...});

    # 启动页面
    page = await browser.newPage();

    # 执行用户交互操作
    ...

    # 等待页面加载完成
    await waitForAnyIdle(page, 30000);

    # 保存并解析trace
    fs.writeFileSync(
      'trace.json',
      JSON.stringify({ traceEvents }, null, 2)
    );
    const result = analyzeTrace(traceEvents);
}
```

在此基础上，由于我们的性能预测模型所预测的目标是：目标参数配置在目标页面下相较于默认配置在目标页面下能达成的加速比而不是绝对的运行时间，因此在 PageExecutor 进行测试的过程中，会针对每个场景额外进行一次默认配置的执行并收集性能信息，以此来获取性能加速比。

5 参数筛选与分析

在完成了测试与执行之后，我们可以基于收集到的参数配置与性能数据进行参数筛选与分析。这一部分的目标是从所有的参数集合 P 中筛选出那些有调优价值的参数集合 P_t，从而减少后续调优的搜索空间，提高调优效率。

我们可以依据参数对性能的影响将参数分为以下几类：

- **无影响参数**：对程序性能几乎没有影响的参数。

USTCCS: S4Plus 张昱课题组

- **稳定影响参数**: 对性能有显著影响，且最优值方向稳定（例如，某个值在所有下都表现最好）。
- **场景敏感参数**: 对性能有显著影响，但最优值随测试场景变化而变化。

对于无影响参数，他们可以被直接排除在调优空间之外。对于稳定影响参数，他们虽然对性能有影响，但是在绝大多数场景下都是保持一个值对性能更好，因此此类参数也不需要通过调优来优化。那么最后，我们只需要关注场景敏感参数，通过调优来调整他们的值，从而获得更好的性能。

为了从节 3中找到的 937 个参数中筛选出有调优价值的参数集合 P_t，我们进行以下分析：1) 首先通过 Classifier 将参数按照其描述进行功能划分。这是因为在所有的参数中，存在很多用于调试、打印、输出等参数，这类参数本身就不是为了性能而添加的，可以通过这种功能划分来将此类参数直接排除在外，得到初步的参数集合 P_c；2) 其次，利用 ImpactAnalyzer，通过实际的实验测试来对 P_c 中的参数进行性能影响分析，得到最终的参数集合 P_t。

5.1 Classifier

图 5展示了参数分类器 Classifier 的设计。首先，通过人工分析将所有的参数分为如表 3中展示的几类，包括用于调试与测试的参数（如 Print,Trace/Debug 等），用于控制 JIT 编译器的参数（如 Turbofan, Maglev 等），用于控制运行时的参数，以及其他的用于控制语言特性（Language mode/features）、用于控制 Wasm 相关的以及一些通用的参数选项等。

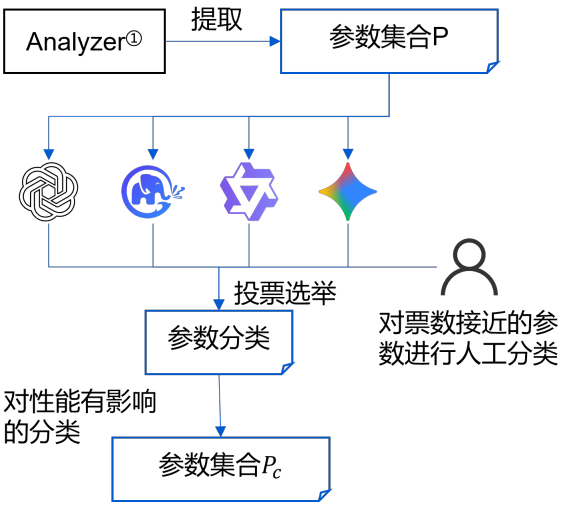

图 5: 参数分类器

其次，基于人工划分的类别，调用多种不同的大模型对参数集合 P 中的参数进行分类，并投票表决，以少数服从多数的原则决定最终的类别。此外，为了保证分类的准确性，还记录了投票的可信度

$$Conf = \frac{\text{投票最高的类别的票数} - \text{投票第二高的类别的票数}}{\text{投票最高的类别的票数}},$$

我们对可信度较低的参数人工进行进一步的分类和辨别，最终得到了表 3中的参数的

分类结果。

在该分类结果中，我们认为和程序运行时性能相关的参数分类包括：GC/Memory, Optimization, Compiler/Tier Manager, Ignition, Sparkplug, Turboshaft, Turbolev, Turbofan, Maglev。上述这些类别的参数共计 440 个。

表 3: 参数分类结果

分类		描述	参数数量
Generic		一些总体上的或通用的参数,例如 experimental 参数表示是否开启实验特性	61
Wasm		WebAssembly 相关	111
Language mode/features		语言模式和语言特性相关	16
调试与测试	Print	信息输出相关	33
	Testing/ Fuzzing	测试相关	82
	Trace/Debug	Trace 或 debug 相关	188
运行时	GC/Memory	垃圾回收和内存管理相关	152
	Optimization	优化机制相关	47
	Data	数据表示相关	13
编译层级	Compiler/ Tier Manager	编译执行框架相关（如 jitless 表示关闭 JIT 编译器，invocation-count-for-maglev 表示需要调用多少次进入到 maglev 编译阶段）	65
	Ignition	Ignition 解释器相关	9
	Sparkplug	Sparkplug 基线编译器相关	14
	Turboshaft	Turboshaft 编译器相关	27
	Turbolev	Turbolev 相关(一个 Turbofan 和 Maglev 结合的 JIT 编译器)	8
	Turbofan	Turbofan 相关	59
	Maglev	Maglev 优化编译器相关	59

USTC-CS-SOFT 张量课题组

5.2 ImpactAnalyzer

ImpactAnalyzer 的核心目标是通过实际的性能测试数据，量化每个参数对程序性能的影响程度及其最优取值的稳定性，从而实现对参数的自动分类。为了隔离单个参数的独立影响，排除与其他参数的潜在关联，这里采用基于随机背景参数配置的测试方法。该方法即通过随机生成若干参数配置（称为背景参数配置），并在上述参数配置中对目标分析参数的值进行变异，从而减少和其他参数关联性。

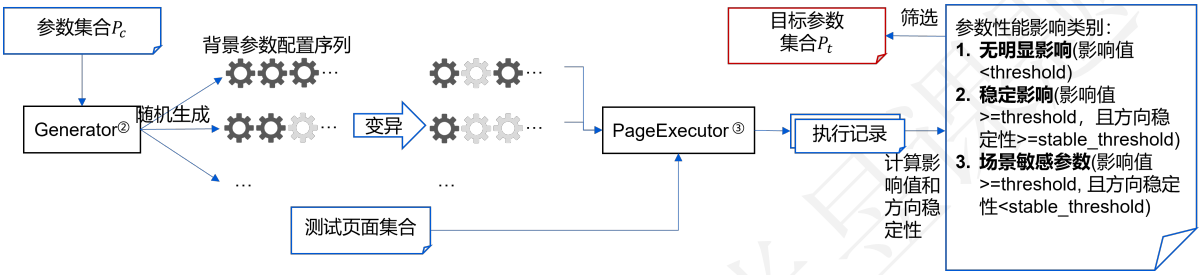

图 6: ImpactAnalyzer

图 6展示了 ImpactAnalyzer 的主要工作流程。它从参数集合 P_c 出发，通过实验数据对参数进行分类。具体而言，ImpactAnalyzer 的工作流程主要包含数据收集、指标计算、参数分类三个步骤，分别介绍如下。

5.2.1 数据收集

为了准确评估参数 p_i 的影响，需要在不同的背景配置下对其进行测试。对于每个待测参数 p_i，首先从除 p_i 以外的所有参数中随机生成 M 组背景配置 $\{c_1, c_2, ..., c_M\}$。然后，针对每个目标测试场景 E 和每一个背景配置 c_i，将参数 p_i 设置为不同的测试值 v（例如布尔类型的 True/False，数值类型的不同倍率值），并重复执行 N 次性能测试。通过对测试结果进行清洗和聚合，我们得到在该具体配置下的稳定性能值 $Perf_{(E,c_i)}(p_i, v)$。同时，为了对比，我们记录了在场景 E 下和背景参数配置 c_i 下，参数 p_i 取默认值时的性能值 $Perf_{(E,c_i)}(p_i, \text{default})$。

5.2.2 指标计算

基于收集到的性能数据，我们定义了两个关键指标来量化参数的特性：

- **影响值 (Impact Value)**：用于衡量参数变化对性能波动的贡献程度。对于特定场景 E 和背景参数配置 c_i，我们计算参数 p_i 在不同取值下的性能相对于基准性

能（默认值）的最大变化率。综合所有背景配置和测试场景，我们得到参数 p_i 的综合影响值：

$$Impact(p_i) = \max_v \{ \frac{|Perf_{(E,c_i)}(p_i, v) - Perf_{(E,c_i)}(p_i, \text{default})|}{Perf_{(E,c_i)}(p_i, \text{default})} \}$$

- **方向稳定性 (Direction Stability)**：用于衡量参数最优取值的一致性。在每个背景配置下，找到使性能最优的参数取值 v_{best}。如果一个参数在大多数背景配置和场景下都倾向于同一个最优值，则认为其方向稳定性较高。稳定性指标 $Stability(P)$ 定义为全局主导最优值出现的频率：

$$Stability(p_i) = \frac{\text{主导最优值出现的次数}}{M \times N}$$

5.2.3 参数分类

最后，根据计算出的综合影响值和方向稳定性，我们将参数划分为三类：

1. **无影响参数**：$Impact(P)$ 低于预设阈值（设置为 0.03），表明该参数对性能影响微乎其微。
2. **稳定影响参数**：$Impact(P)$ 高于阈值，且 $Stability(P)$ 高于稳定性阈值（设置为 0.85）。这类参数虽然影响性能，但其最优值往往固定（例如始终关闭某个特性性能最好），因此无需在调优过程中反复搜索。
3. **场景敏感参数**：$Impact(P)$ 高于阈值，但 $Stability(P)$ 低于稳定性阈值。这类参数的最优取值随场景和背景配置变化较大，是自动调优工具需要重点关注的对象。

通过上述分析流程，ImpactAnalyzer 能够有效地从大量参数中筛选出真正具有调优价值的“场景敏感参数”，构建最终的调优参数集合 P_t。

5.2.4 分类结果

图 7展示了在所有 440 个 P_c 集合参数中，影响值和方向稳定性的分布情况。可以看到，大部分参数的影响值都低于 0.037，稳定性也大多低于 0.85。最终，我们通过将影响值的阈值设置为 0.03，稳定性阈值设置为 0.85，得到了最终的参数集合 P_t，共包含 171 个不同的参数。

在我们的实验中，对性能影响最显著的几个参数包括：use-ic（控制是否开启 inline cache），该参数的影响值最高为 0.7017，而其稳定性为 1，主导值为 true，说明开启 inline cache 对性能有显著的提升。maglev-as-top-tier （控制是否将 maglev 作为 JIT 编译流水中最顶级的编译器，而不再进入到 Turbofan）的影响值排名第二，并且方向

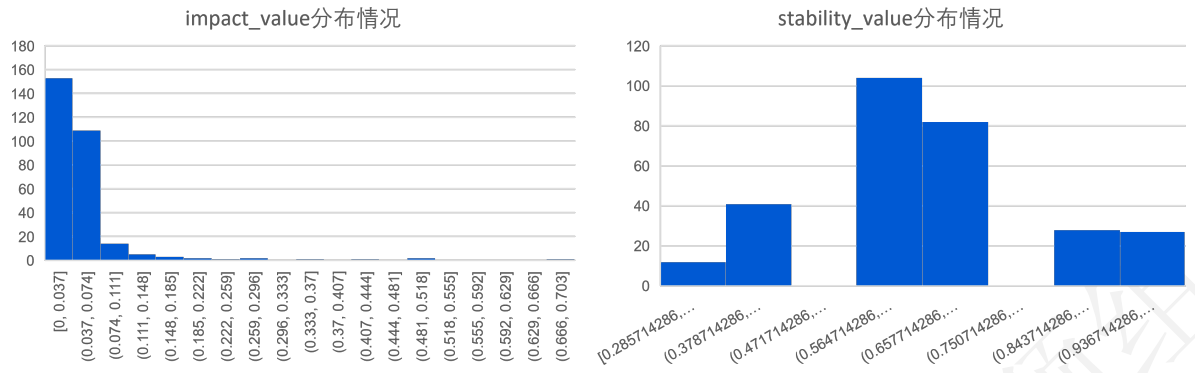

图 7: 参数分类结果

稳定性为 1，主导值则相反为 false，这意味着关闭该选项在绝大多数情况下对性能有益。此外，还有一些参数如 max-valid-polymorphic-map-count（控制多态内联缓存所能记录的状态数量）也表现出了对性能的明显影响，作为一个整型参数，它在不同场景下的性能最优值都不同，这意味着它是我们调优器所应该重点关注的参数。

6 总结与未来工作

在本阶段，我们首先通过模式匹配的脚本提取了 V8 虚拟机参数的信息与依赖关系；其次，利用 Puppeteer 等工具设计了用于进行实证实验数据收集的测试框架，并通过对 V8 的修改实现了特征信息的提取；最后，我们通过调用测试框架收集数据，在实证数据上对参数的功能和影响性进行了分析，最终筛选出了用于后续性能调优的参数集合。

目前进行的分析和结果主要是基于 V8 源码和实现得来的，在后续的工作中，我们将根据调优器的需求来进一步改善和调整我们的分析方法和结果，包括：

- 调整特征集合。当前阶段选择的特征信息是一种基于人工分析和专家经验的选择，并不一定能很好地对不同场景下的 JS 程序进行表征，在后续的模型训练的过程中，我们需要根据模型训练的结果对特征信息的选择进行调整。
- 特征集合的降维。不同特征对于程序表征的贡献度不同，可以尝试通过主成分分析等方式来对于一些重复表征的信息进行降维等。
- 参数的隐式关联的进一步分析。当前的分析结果中，我们从源码中提取了参数的显式依赖关系，但是参数之间可能还存在一些隐式的关联关系，例如可能存在某些参数他们通常在同时设置为一定值的时候能达到更好的效果。

USTC-OS SMPlus 收集课题组

A 影响值 >0.03 的参数集合

名称	类型	默认值	类别	描述	影响值	稳定性	主导最优值
use-ic	boolean	TRUE	Optimization	use inline caching	0.701733	1	TRUE
maglev-as-top-tier	boolean	FALSE	Maglev	disable Turbofan/ Turbolev, make Maglev optimize more aggressively	0.491761	1	FALSE
turboshaft	boolean	TRUE	Turboshaft	enable TurboFan's Turboshaft phases for JS	0.487182	0.857143	TRUE
gc-global	boolean	FALSE	GC/ Memory	always perform global GCs	0.407451	0.857143	FALSE
max-valid-polymorphic-map-count	integer	4	Optimization	maximum number of valid maps to track in POLYMORPHIC state	0.366575	0.285714	4
turbo-inlining	boolean	TRUE	Turbofan	enable inlining in TurboFan	0.277344	0.714286	TRUE
concurrent-recompilation	boolean	TRUE	Compiler/ Tier Manager	optimizing hot functions asynchronously on a separate thread	0.277133	0.857143	TRUE
lite-mode	boolean	FALSE	GC/ Memory	enables trade-off of performance for memory savings	0.224381	1	FALSE
disable-optimizing-compilers	boolean	FALSE	Compiler/ Tier Manager	Disable all optimizing compilers while leaving baseline compilers enabled	0.219735	1	FALSE
maglev-loop-peeling-max-size	integer	400	Maglev	max loop size for loop peeling in the maglev optimizing compiler	0.209902	0.285714	200
minimum-invocations-after-ic-update	integer	500	Compiler/ Tier Manager	How long to minimally wait after IC update before tier up	0.173559	0.428571	250
incremental-marking-hard-trigger	integer	0	GC/ Memory	threshold for starting incremental marking immediately in percent of available space: limit - size	0.170689	0.428571	0
gc-memory-reducer-start-delay-ms	integer	8000	GC/ Memory	Delay before memory reducer start	0.15419	0.428571	8000
maglev-loop-peeling-max-size-cumulative	integer	900	Maglev	max cumulative size for loop peeling in the maglev optimizing compiler	0.13893	0.428571	900
inline-new	boolean	TRUE	Optimization	use fast inline allocation	0.119929	1	TRUE

USTC-CS: SAPLIS 共享课题组

max-old-space-size	integer	0	GC/ Memory	max size of the old space (in Mbytes)	0.117615	0.714286	0
turbolev	boolean	FALSE	Turbolev	use Turbolev (F?Maglev + Turboshaft combined) as the 4th tier compiler instead of Turbofan	0.114523	0.571429	TRUE
reopt-after-lazy-deopts	boolean	TRUE	Optimization	Immediately re-optimize code after some lazy deopts	0.112786	0.714286	TRUE
invocation-count-for-maglev	integer	400	Compiler/ Tier Manager	invocation count required for optimizing with Maglev	0.110949	0.714286	200
maglev-assert-stack-size	boolean	TRUE	Maglev	insert stack size checks before every IR node	0.108716	0.857143	FALSE
maglev-osr	boolean	TRUE	Maglev	use maglev as on-stack replacement target	0.095393	0.714286	TRUE
max-inlined-bytecode-size-cumulative	integer	920	Turbofan	maximum cumulative size of bytecode considered for inlining	0.094661	0.571429	1840
max-semi-space-size	integer	0	GC/ Memory	max size of a semi-space (in MBytes), the new space consists of two semi-spaces	0.093908	0.571429	0
maglev-overwrite-budget	boolean	FALSE	Maglev	whether maglev resets the interrupt budget	0.092271	0.571429	FALSE
reserve-inline-budget-scale-factor	float	1.2	Turbofan	scale factor of bytecode size used to calculate the inlining budget	0.090028	0.714286	0.6
turbo-load-elimination	boolean	TRUE	Turbofan	enable load elimination in TurboFan	0.088834	0.714286	TRUE
sparkplug	boolean	TRUE	Sparkplug	enable Sparkplug baseline compiler	0.086667	0.857143	TRUE
max-maglev-inline-depth	integer	1	Maglev	max depth of functions that Maglev will inline excl. small functions	0.081023	0.428571	2
concurrent-recompilation-delay	integer	0	Compiler/ Tier Manager	artificial compilation delay in ms	0.078974	0.428571	0
lazy-compile-dispatcher-max-threads	integer	0	Compiler/ Tier Manager	max threads for compiler dispatcher (0 for unbounded)	0.078179	0.428571	0

USTC CS: SAFTRS 张宝课题组

invocation-count-for-turbofan	integer	3000	Compiler/ Tier Manager	invocation count required for optimizing with TurboFan	0.077745	0.714286	1500
ignition-reo	boolean	TRUE	Ignition	use ignition register equivalence optimizer	0.076433	0.714286	TRUE
max-maglev-inlined-bytecode-size-small	integer	27	Maglev	maximum size of bytecode considered for small function inlining	0.073607	0.857143	13
parallel-pointer-update	boolean	TRUE	GC/ Memory	use parallel pointer update during compaction	0.073421	0.714286	FALSE
analyze-environment-liveness	boolean	TRUE	Optimization	analyze liveness of environment slots and zap dead values	0.072878	1	TRUE
maglev-inline-date-accessors	boolean	FALSE	Maglev	Inline Date accessors by Maglev	0.069827	0.714286	TRUE
preconfigured-old-space-size	integer	0	GC/ Memory	preconfigured old space size (in Mbytes)	0.069352	0.428571	0
max-inlined-bytecode-size-small	integer	27	Turbofan	maximum size of bytecode considered for small function inlining	0.068964	0.428571	13
osr-from-maglev	boolean	FALSE	Compiler/ Tier Manager	whether we try to OSR to Turbofan from OSR'd Maglev	0.06877	0.714286	FALSE
invocation-count-for-maglev-with-delay	integer	600	Compiler/ Tier Manager	invocation count for maglev for functions which according to profile_guided_optimization are likely to deoptimize before reaching this invocation count	0.066157	0.428571	300
concurrent-turbofan-max-threads	integer	4	Turbofan	max number of threads that concurrent Turbofan can use (0 for unbounded)	0.06542	0.428571	8
lazy-feedback-allocation	boolean	TRUE	Optimization	Allocate feedback vectors lazily	0.063817	0.714286	FALSE
invocation-count-for-maglev-osr	integer	100	Compiler/ Tier Manager	invocation count required for maglev OSR	0.062211	0.285714	50
minimum-invocations-before-optimization	integer	2	Compiler/ Tier Manager	Minimum number of invocations we need before non-OSR optimization	0.060668	0.428571	1

USTC-CS SAPLUS 张星课题组

maglev-escape-analysis	boolean	TRUE	Maglev	avoid inlined allocation of objects that cannot escape	0.060575	0.571429	TRUE
initial-old-space-size	integer	0	GC/ Memory	initial old space size (in Mbytes)	0.060246	0.428571	1
min-semi-space-size	integer	0	GC/ Memory	min size of a semi-space (in MBytes), the new space consists of two semi-spaces	0.059276	0.428571	4
max-maglev-inlined-bytecode-size	integer	100	Maglev	maximum size of bytecode for a single inlining	0.058363	0.428571	150
max-inlined-bytecode-size-small-total	integer	30000	Turbofan	maximum size of small functions' bytecode that can be inlining	0.057345	0.428571	15000
heap-growing-percent	integer	0	GC/ Memory	specifies heap growing factor as (1 + heap_growing_percent/ 100)	0.056948	0.571429	2
lazy-new-space-shrinking	boolean	FALSE	GC/ Memory	Enables the lazy new space shrinking strategy	0.055935	0.571429	TRUE
always-osr	boolean	FALSE	Compiler/ Tier Manager	always try to OSR functions	0.055919	0.571429	TRUE
min-maglev-inlining-frequency	float	0.95	Maglev	minimum frequency for inlining	0.055589	0.428571	1.425
turbo-jt	boolean	TRUE	Turbofan	enable jump threading in TurboFan	0.055095	0.857143	TRUE
max-opt	integer	999	Compiler/ Tier Manager	Set the maximal optimisation tier: > 3 == any, 0 == ignition/interpreter, 1 == sparkplug/baseline, 2 == maglev, 3 == turbofan	0.054581	0.428571	999
baseline-batch-compilation	boolean	TRUE	Sparkplug	batch compile Sparkplug code	0.052938	0.571429	TRUE
external-memory-max-growing-factor	float	1.3	GC/ Memory	When external memory limits are computed as poart of the global limits,this is the upper bound for growing factor imposed on external memory.	0.052764	0.571429	0.65
maglev-inlining	boolean	TRUE	Maglev	enable inlining in the maglev optimizing compiler	0.052737	0.714286	FALSE

USTC-CS: S4Plus 毕业课题组

minor-ms-min-lab-size-kb	integer	0	GC/ Memory	override for the minimum lab size in KB to be used for new space allocations with minor ms.	0.052614	0.428571	2
fast-properties-soft-limit	integer	12	Optimization	limits the number of properties that can be added to an object using keyed store before transitioning to dictionary mode	0.052107	0.285714	18
incremental-marking-soft-trigger	integer	0	GC/ Memory	threshold for starting incremental marking via a task in percent of available space: limit - size	0.051913	0.428571	4
max-optimized-bytecode-size	integer	61440	Turbofan	maximum bytecode size to be considered for turbofan optimization; too high values may cause the compiler to hit (release) assertions	0.05157	0.285714	61440
page-promotion-threshold	integer	70	GC/ Memory	min percentage of live bytes on a page to enable fast evacuation in full GCs	0.051484	0.428571	140
minor-ms-concurrent-marking-trigger	integer	90	GC/ Memory	minor ms concurrent marking trigger in percent of the current new space capacity	0.051165	0.428571	45
minor-ms-page-promotion-threshold	integer	50	GC/ Memory	min percentage of live bytes on a page to enable fast evacuation in MinorMS	0.051005	0.285714	75
turbo-optimize-math-minmax	boolean	TRUE	Turbofan	optimize call math.min/ max with double array	0.050789	0.571429	FALSE
osr-from-maglev-interrupt-scale-factor	float	0.8	Compiler/ Tier Manager	Scale interrupt budget reduction for OSR from Maglev vs. OSR to Maglev	0.050753	0.714286	1.2
concurrent-marking-high-priority-threads	boolean	FALSE	GC/ Memory	use high priority threads for concurrent Marking	0.050715	0.571429	FALSE
memory-balancer-c-value	float	#####	GC/ Memory	c value for membalancer. A special constant to balance between memory and space tradeoff. The smaller the more memory it uses.	0.050526	0.571429	4.50E-10
bytecode-old-time	integer	30	GC/ Memory	number of seconds before we flush code	0.050431	0.428571	60

USTC-CS: SAPLab 张昱课题组

invocation-count-for-feedback-allocation	integer	8	Compiler/ Tier Manager	invocation count required for allocating feedback vectors	0.049937	0.571429	4
minor-gc-task-trigger	integer	80	GC/ Memory	minor GC task trigger in percent of the current heap limit	0.049931	0.428571	40
switch-table-spread-threshold	integer	3	Optimization	allow the jump table used for switch statements to span a range of integers roughly equal to this number times the number of clauses in the switch	0.049817	0.285714	3
max-turbolev-inlined-bytecode-size-cumulative	integer	1840	Turbolev	maximum cumulative size of bytecode considered for inlining	0.048882	0.571429	920
compact-on-every-full-gc	boolean	FALSE	GC/ Memory	Perform compaction on every full GC	0.048572	0.857143	TRUE
page-pool-timeout	integer	8	GC/ Memory	Release pooled pages after X seconds.	0.048367	0.285714	8
ineffective-gc-mutator-utilization-threshold	float	0.4	GC/ Memory	Threshold on mutator utilization to trigger out-of-memory failure near heap limit.	0.04827	0.571429	0.8
concurrent-recompilation-queue-length	integer	8	Compiler/ Tier Manager	the length of the concurrent compilation queue	0.048223	0.571429	12
osr-to-tierup	integer	1	Compiler/ Tier Manager	number to decrease the invocation budget by when we follow OSR	0.047632	0.571429	2
memory-reducer	boolean	TRUE	GC/ Memory	use memory reducer	0.047619	0.857143	TRUE
concurrent-maglev-max-threads	integer	2	Maglev	max number of threads that concurrent Maglev can use (0 for unbounded)	0.047554	0.428571	1
invocation-count-for-early-optimization	integer	30	Compiler/ Tier Manager	invocation count threshold for early optimization	0.047404	0.571429	45
turbo-loop-peeling	boolean	TRUE	Turbofan	TurboFan loop peeling	0.046744	0.571429	FALSE
scavenger-chaos-mode-threshold	integer	50	GC/ Memory	Percentage of non-large young objects that will be promoted during Scavenger in chaos mode	0.046724	0.428571	75

USTC-CS SMPlus 张昱课题组

compact-with-stack	boolean	TRUE	GC/ Memory	Perform compaction when finalizing a full GC with stack	0.046693	0.571429	FALSE
max-large-page-pool-size	integer	32	GC/ Memory	Maximum size of pooled large pages in MB.	0.046691	0.428571	64
minor-ms-min-new-space-capacity-for-concurrent-marking-mb	integer	0	GC/ Memory	min new space capacity in MBs for using young generation concurrent marking.	0.046668	0.428571	4
optimize-for-size	boolean	FALSE	Optimization	Enables optimizations which favor memory size over execution speed	0.046614	1	FALSE
max-maglev-inlined-bytecode-size-small-with-heapnum-in-out	integer	75	Maglev	maximum size of bytecode considered for small function inlining, when the function call has a HeapNumber as input or output	0.046444	0.571429	150
initial-heap-size	integer	0	GC/ Memory	initial size of the heap (in Mbytes)	0.046035	0.857143	0
maglev-destroy-on-background	boolean	TRUE	Maglev	Destroy compilation jobs on background thread	0.045817	0.857143	TRUE
invocation-count-for-osr	integer	500	Compiler/ Tier Manager	invocation count required for OSR	0.045422	0.428571	750
optimize-gc-for-battery	boolean	FALSE	GC/ Memory	optimize GC for battery	0.045357	0.571429	FALSE
ineffective-gc-size-threshold	float	0.8	GC/ Memory	Threshold on heap size to trigger out-of-memory failure near heap limit.	0.044743	0.571429	0.8
minor-ms-max-page-age	integer	4	GC/ Memory	max age for a page after which it is force promoted to old space	0.044091	0.571429	4
semi-space-growth-factor	integer	2	GC/ Memory	factor by which to grow the new space	0.044079	0.714286	4
max-shared-heap-size	integer	0	GC/ Memory	max size of the shared heap (in Mbytes); other heap size flags (e.g. max_shared_heap_size) take precedence	0.043981	0.428571	2
always-sparkplug	boolean	FALSE	Sparkplug	directly tier up to Sparkplug code	0.043688	0.571429	TRUE

USTC-CS-SAPlus 张亚课题组

empty-context-extension-dep	boolean	TRUE	Optimization	Use compilation dependency to avoid dynamic checks for non-empty context extensions	0.043637	0.571429	FALSE
parallel-scavenge	boolean	TRUE	GC/ Memory	parallel scavenge	0.043423	0.714286	FALSE
max-inlined-bytecode-size-absolute	integer	4600	Turbofan	maximum absolute size of bytecode considered for inlining	0.043173	0.571429	6900
reclaim-unmodified-wrappers	boolean	TRUE	GC/ Memory	reclaim otherwise unreachable unmodified wrapper objects when possible	0.043134	0.714286	TRUE
ephemeron-fixpoint-iterations	integer	10	GC/ Memory	number of fixpoint iterations it takes to switch to linear ephemeron algorithm	0.042863	0.428571	15
turbo-inline-js-wasm-calls	boolean	TRUE	Turbofan	inline JS->Wasm calls (specifically: inline JS-to-Wasm wrappers and then the body of the Wasm function, if applicable)	0.042851	0.571429	FALSE
max-inlined-bytecode-size-small-with-heapnum-in-out	integer	75	Turbofan	maximum size of bytecode considered for small function inlining, when the function call has a HeapNumber as input or output	0.042798	0.571429	112
function-context-cells-max-size	integer	1	Optimization	Max number of context slots allowed in a function context with context cells	0.04226	0.571429	0
max-maglev-hard-inline-depth	integer	10	Maglev	max depth of functions that Maglev will inline incl. small functions	0.041813	0.285714	10
efficiency-mode-disable-turbofan	boolean	FALSE	Compiler/ Tier Manager	Defer tier-up to turbofan while in efficiency mode.	0.041587	0.571429	TRUE
concurrent-sparkplug-max-threads	integer	1	Sparkplug	max number of threads that concurrent Sparkplug can use (0 for unbounded)	0.041442	0.571429	0
memory-reducer-respects-frozen-state	boolean	FALSE	GC/ Memory	don't schedule another GC when we are frozen	0.041413	0.571429	FALSE
gc-interval	integer	-1	GC/ Memory	garbage collect after <n> allocations	0.041389	0.571429	0

USTC-CS: SAPlus 实星课题组

gc-stats	integer	0	GC/ Memory	Used by tracing internally to enable gc statistics	0.041367	0.571429	2
concurrent-marking-max-worker-num	integer	7	GC/ Memory	max worker number of concurrent marking, 0 for NumberOfWorkerThreads	0.041299	0.428571	3
verify-heap-skip-remembered-set	boolean	FALSE	GC/ Memory	disable remembered set verification	0.041288	0.571429	FALSE
min-inlining-frequency	float	0.15	Turbofan	minimum frequency for inlining	0.041177	0.285714	0.075
retain-maps-for-n-gc	integer	2	GC/ Memory	keeps maps alive for <n> old space garbage collections	0.041116	0.428571	4
turboshaft-enable-debug-features	boolean	FALSE	Turboshaft	enables Turboshaft's DebugPrint, StaticAssert and CheckTurboshaftTypeOf operations	0.041076	0.571429	TRUE
baseline-batch-compilation-threshold	integer	4096	Sparkplug	the estimated instruction size of a batch to trigger compilation	0.040516	0.428571	6144
turboshaft-wasm-in-js-inlining	boolean	FALSE	Turboshaft	inline Wasm code into JS functions via Turboshaft (instead of via TurboFan). Only the Wasm code is inlined in Turboshaft, the JS-to-Wasm wrappers are still inlined in TurboFan. For controlling whether to inline at all, see –turbo-inline-js-wasm-calls. (experimental)	0.040464	0.714286	FALSE
feedback-normalization	boolean	FALSE	Optimization	feed back normalization to constructors (experimental)	0.040226	0.571429	FALSE
max-inlined-bytecode-size	integer	460	Turbofan	maximum size of bytecode for a single inlining	0.040074	0.285714	460
maglev-allocation-folding	integer	2	Maglev	maglev allocation folding level	0.040052	0.428571	1
profile-guided-optimization	boolean	TRUE	Optimization	profile guided optimization	0.039556	0.857143	FALSE
specialize-code-for-one-byte-seq-strings	boolean	FALSE	Maglev	Specialize maglev code for feedback with only one-byte sequential strings. (experimental)	0.039504	0.571429	FALSE

USTC CS: SAPlus 荣誉课题组

flush-code-based-on-time	boolean	FALSE	Compiler/ Tier Manager	Use time-base code flushing instead of age.	0.039367	0.714286	FALSE
bytecode-old-age	integer	6	GC/ Memory	number of gcs before we flush code	0.038748	0.428571	6
efficiency-mode-delay-turbofan-multiply	integer	3	Compiler/ Tier Manager	Delay tier-up to turbofan to a certain invocation count multipier while in efficiency mode.	0.038596	0.428571	4
max-lazy	boolean	FALSE	Compiler/ Tier Manager	ignore eager compilation hints	0.038504	0.571429	FALSE
external-memory-accounted-in-global-limit	boolean	FALSE	GC/ Memory	External memory limits are computed as part of global limits in v8 Heap.	0.038412	0.857143	TRUE
max-maglev-inlined-bytecode-size-cumulative	integer	920	Maglev	maximum cumulative size of bytecode considered for inlining excl. small functions	0.038402	0.571429	1380
turboshaft-load-elimination	boolean	TRUE	Turboshaft	enable Turboshaft's low-level load elimination for JS	0.038401	0.571429	TRUE
memory-pool-release-before-memory-pressure-gcs	boolean	TRUE	GC/ Memory	discard the memory pool before invoking the GC on memory pressure or last resort GCs	0.038293	0.571429	FALSE
regexp-assemble-from-bytecode	boolean	FALSE	Compiler/ Tier Manager	assemble regexp JIT-code from bytecode	0.038269	0.714286	FALSE
switch-table-min-cases	integer	6	Optimization	the number of Smi integer cases present in the switch statement before using the jump table optimization	0.038249	0.428571	9
maglev-speculative-hoist-phi-untagging	boolean	FALSE	Maglev	enable phi untagging to hoist untagging of loop phi inputs (could still cause deopt loops) (experimental)	0.03824	0.571429	TRUE
max-fast-properties	integer	128	Optimization	limits the number of mutable properties that can be added to an object before transitioning to dictionary mode	0.038201	0.285714	128

USTC-CS: SAPlus 张晨课题组

maglev-inline-api-calls	boolean	FALSE	Maglev	Inline CallApiCallback builtin into generated code	0.038136	0.714286	TRUE
regexp-tier-up-ticks	integer	1	Compiler/ Tier Manager	set the number of executions for the regexp interpreter before tiering-up to the compiler	0.037673	0.714286	1
profile-guided-optimization-for-empty-feedback-vector	boolean	TRUE	Optimization	profile guided optimization for empty feedback vector	0.037318	0.714286	TRUE
turbo-allocation-folding	boolean	TRUE	Turbofan	TurboFan allocation folding	0.037147	0.714286	TRUE
turbolev-non-eager-inlining	boolean	FALSE	Turbolev	enable Turbolev non-eager inlining (experimental)	0.03708	0.714286	FALSE
initial-shared-heap-size	integer	0	GC/ Memory	initial size of the shared heap (in Mbytes); other heap size flags (e.g. initial_heap_size) take precedence	0.036966	0.428571	1
turbo-optimize-inlined-js-wasm-wrappers	boolean	FALSE	Turbofan	Run additional optimizations (especially load-elimination) on inlined JS-to-Wasm wrappers	0.036695	0.857143	TRUE
idle-gc-on-context-disposal	boolean	TRUE	GC/ Memory	idle gc on context disposal	0.036549	0.714286	FALSE
turbo-store-elimination	boolean	TRUE	Turbofan	enable store-store elimination in TurboFan	0.036373	0.857143	TRUE
lazy-streaming	boolean	TRUE	Compiler/ Tier Manager	use lazy compilation during streaming compilation	0.036246	0.571429	TRUE
shortcut-strings-with-stack	boolean	TRUE	GC/ Memory	Shortcut Strings during GC with stack	0.036231	0.714286	TRUE
memory-reducer-gc-count	integer	2	GC/ Memory	Maximum number of memory reducer GCs scheduled	0.035708	0.428571	1
minor-ms-page-promotion-max-lab-threshold	integer	30	GC/ Memory	max percentage of labs out of a page to still be considered for page promotion	0.035554	0.428571	60

USTC-CS-SAPLab 张昱课题组

parallel-pause-for-gc-in-background	boolean	TRUE	GC/ Memory	Use parallel threads in the atomic pause for background GCs	0.035537	0.714286	FALSE
deduplicate-heap-number-requests	boolean	FALSE	Optimization	avoid duplicated HeapNumber materializations in HeapNumberRequests	0.035189	0.857143	FALSE
mega-dom-ic	boolean	FALSE	Optimization	use MegaDOM IC state for API objects	0.035174	0.571429	FALSE
max-maglev-inlined-bytecode-size-small-total	integer	3000	Maglev	maximum size of small functions' bytecode that can be inlining	0.035056	0.571429	6000
decommit-pooled-pages	boolean	TRUE	GC/ Memory	decommit, rather than discard pooled pages	0.034641	0.571429	FALSE
concurrent-osr	boolean	TRUE	Compiler/ Tier Manager	enable concurrent OSR	0.034535	0.714286	FALSE
scavenger-promote-quarantined-pages	boolean	TRUE	GC/ Memory	Quarantined pages in the intermediate generation will be promoted to old space	0.034446	0.714286	TRUE
maglev-object-tracking	boolean	FALSE	Maglev	track object changes to avoid escaping them (experimental)	0.034383	0.714286	TRUE
transition-strings-during-gc-with-stack	boolean	FALSE	GC/ Memory	Transition strings during a full GC with stack	0.034178	0.571429	FALSE
lazy-compile-dispatcher	boolean	FALSE	Compiler/ Tier Manager	enable compiler dispatcher (experimental)	0.033956	0.714286	TRUE
maglev-loop-peeling	boolean	TRUE	Maglev	enable loop peeling in the maglev optimizing compiler	0.033866	0.571429	FALSE
incremental-marking-unified-schedule	boolean	FALSE	GC/ Memory	Use a single schedule for determining a marking schedule between JS and C++ objects.	0.033827	0.714286	FALSE
single-threaded-gc-in-background	boolean	TRUE	GC/ Memory	disable the use of background gc tasks when in background	0.033788	0.857143	FALSE

USTC.CS.SAPLab 张昱课题组

turbo-instruction-scheduling	boolean FALSE	Turbofan	enable instruction scheduling in TurboFan	0.033778	0.571429	FALSE
flush-bytecode	boolean TRUE	GC/ Memory	flush of bytecode when it has not been executed recently	0.033123	0.714286	TRUE
memory-pool	boolean TRUE	GC/ Memory	Pool memory globally before returning it to the operating system	0.033117	0.714286	FALSE
memory-reducer-for-small-heaps	boolean TRUE	GC/ Memory	use memory reducer for small heaps	0.033093	0.857143	TRUE
enable-regexp-unaligned-accesses	boolean TRUE	Optimization	enable unaligned accesses for the regexp engine	0.032851	0.571429	FALSE
concurrent-sweeping	boolean TRUE	GC/ Memory	use concurrent sweeping	0.032816	1	TRUE
parallel-reclaim-unmodified-wrappers	boolean TRUE	GC/ Memory	reclaim wrapper objects in parallel	0.032574	0.714286	TRUE
regexp-tier-up	boolean TRUE	Compiler/ Tier Manager	enable regexp interpreter and tier up to the compiler after the number of executions set by the tier up ticks flag	0.032504	0.714286	TRUE
track-gc-object-stats	boolean FALSE	GC/ Memory	track object counts and memory usage	0.032493	0.714286	TRUE
concurrent-cache-deserialization	boolean TRUE	Compiler/ Tier Manager	enable deserializing code caches on background	0.032268	0.571429	FALSE
parallel-weak-ref-clearing	boolean TRUE	GC/ Memory	use parallel threads to clear weak refs in the atomic pause.	0.032249	0.571429	FALSE
ineffective-gc-includes-global	boolean FALSE	GC/ Memory	includes global size in out-of-memory failure near heap limit	0.031625	0.571429	FALSE
turboshaft-typed-optimizations	boolean FALSE	Turboshaft	enable an additional Turboshaft phase that performs optimizations based on type information (experimental)	0.031555	0.571429	TRUE
always-osr-from-maglev	boolean FALSE	Compiler/ Tier Manager	whether we try to OSR to Turbofan from any Maglev	0.031498	1	FALSE
script-context-cells	boolean TRUE	Optimization	Use context cells in script contexts, ie, const tracking let and mutable numbers	0.031172	0.857143	TRUE

USTC-CS SAPIS 张量课题组

enable-lazy-source-positions	boolean	TRUE	Compiler/ Tier Manager	skip generating source positions during initial compile but regenerate when actually required	0.030645	0.714286	FALSE
concurrent-sparkplug-high-priority-threads	boolean	FALSE	Sparkplug	use high priority compiler threads for concurrent Sparkplug	0.030528	0.571429	FALSE
maglev-range-analysis	boolean	FALSE	Maglev	Enable Maglev range value analysis pass (experimental)	0.030329	0.571429	FALSE

USTC-CS: S4Plus 张昱课题组

参考文献

[1] Jie Ren, Ling Gao, and Zheng Wang. Javascript performance tuning as a crowdsourced service. IEEE Transactions on Mobile Computing, 23(5):6116–6132, 2024.

USTC-CS: S4Plus 张昱课题组